



- +91-7081181585
- CONTACT@SYNTHERAROBOTICS.IN
- NAWADA , NEW DELHI,110059

MOBILE APP DEVELOPMENT ASSIGNMENT

Embedded Systems & Mobile App Development Intern

Build a Mobile App for a Simulated Prosthetic Hand

1. Assignment Objective

Synthera Robotics is developing ESP32-based robotic and assistive-technology systems. For this assignment, you will develop a mobile application that acts as the control and monitoring interface for a prosthetic hand.

You do NOT need an ESP32, prosthetic hand, or any physical hardware. Create a software simulation of the device inside your application. We are evaluating your mobile development, UI/UX, application architecture, real-time data handling, hardware–software integration thinking, and problem-solving.

2. Dashboard

Create a clean, professional dashboard displaying:

- Device name
- Connection status
- Battery percentage
- Hand position
- Current operating mode
- EMG signal/value
- Current hand state

```
SYNTHERA ROBOTICS
DAKSH-01
Status: Connected
Battery: 82%
Hand Position: 45°
State: HOLDING
EMG Signal: 127
Mode: AUTO
[ OPEN ] [ CLOSE ] [ STOP ]
```

3. Simulated Prosthetic Hand

Implement a software simulation of the prosthetic hand.

- Fully open position: 0°
- Fully closed position: 63°
- OPEN should smoothly move the simulated position from its current angle toward 0°.
- CLOSE should smoothly move the simulated position from its current angle toward 63°.
- STOP should immediately stop simulated movement.
- Represent movement visually using animation, a hand graphic, progress indicator, or another clear visualization.

4. Simulated Device Communication

Because no hardware is required, implement a simulated communication layer representing how the mobile application would communicate with an ESP32.

```
Mobile App
  ↓
Device / Communication Service
  ↓
Simulated ESP32
  ↓
Simulated Prosthetic Hand
```

The simulated device should accept commands such as:

```
OPEN    |    CLOSE    |    STOP    |    CALIBRATE
```

and return simulated device data such as:

```
BATTERY:82    POSITION:45    EMG:127    MODE:AUTO    STATE:HOLDING
```

Structure the code so that the simulated communication layer could later be replaced by a real BLE/ESP32 implementation.

5. Real-Time Data Simulation

Simulate changing sensor data.

- EMG value should continuously change within a sensible range.
- Display the EMG value in the application.
- Bonus: display the EMG signal as a real-time graph.

```
Example EMG values: 50 → 83 → 124 → 180 → 142 → 96 → 61 ...
```

6. Battery Simulation

Simulate battery data decreasing gradually while the simulated device is operating. Display a low-battery warning when the battery reaches the configured warning threshold.

```
⚠ LOW BATTERY
Battery: 18%
```

7. Operating Modes

Implement at least three modes:

Manual: User directly controls OPEN, CLOSE and STOP.

EMG: Simulate EMG-based control. For example, a contraction can trigger closing and a subsequent contraction can trigger opening.

Auto: Simulate predefined hand movements.

8. Settings

Create a settings page containing editable values such as:

- Device name
- Minimum angle
- Maximum angle
- EMG threshold
- Battery warning threshold
- Operating mode

Save the settings locally or through your application state where appropriate.

9. Calibration

Create a basic calibration workflow:

```
Step 1 - Move hand to OPEN position
Step 2 - Move hand to CLOSED position
Step 3 - Save calibration
Result - Calibration Complete
```

The application should retain the calibrated minimum and maximum positions.

10. Error Handling

- Device disconnected
- Connection/reconnection failure
- Invalid command
- Low battery
- Invalid or unavailable sensor data
- Emergency STOP

```
Device Disconnected
[ RECONNECT ]
```

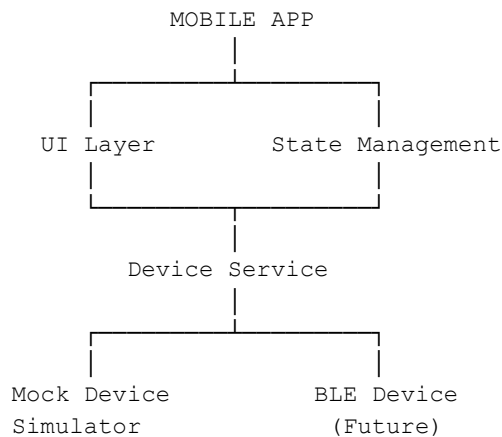
11. Technical Requirements

Preferred framework: Flutter / Dart.

Also accepted: React Native, Kotlin/Android, Java/Android, or another suitable mobile framework.

You may use state-management, animation, chart, local-storage, mock-API, and other appropriate libraries. Explain your technology choices in the README.

12. Recommended Architecture



Only the Mock Device Simulator is required for this assignment. The architecture should make future replacement with real ESP32/BLE communication straightforward.

13. Deliverables

Submit the following:

- GitHub Repository Link — complete source code and README.
- Live App / View / Demo Link — a working hosted application, web build, APK/demo page, or other accessible view of the project.

Your README should include:

- Project overview
- Technologies used
- Application architecture
- How the simulated device works
- How you would connect the application to ESP32/BLE
- Installation and run instructions
- Screenshots
- Known limitations

Recommended demo: a 2–5 minute video showing the main features.

14. Submission Format

Submission Item	Link
GitHub Repository	[Paste GitHub link]
View / Live Demo	[Paste view/demo link]

Submit links only. No physical hardware is required.

15. Time Limit

Recommended completion time: 3–5 days.

The assignment can be completed using a laptop and a mobile emulator or physical smartphone. No ESP32 or prosthetic hardware is required.

16. Evaluation Criteria

Category	Weight
UI/UX & App Design	20%
Application Functionality	25%
Simulation & Real-Time Data	15%
Code Architecture	15%
Hardware/BLE Understanding	10%
Error Handling	5%
Documentation	5%
Innovation / Bonus Features	5%
TOTAL	100%

17. Bonus Features

- Real-time EMG graph
- Smooth hand animation
- Dark/light mode
- Local data storage
- Device logs
- Automatic reconnection simulation
- Multiple-device support
- OTA update interface
- User authentication
- Clean BLE abstraction
- API integration
- Excellent UI/UX

18. What We Are Looking For

We are not expecting a production-ready prosthetic application. We want to understand how you think as a developer particularly how you design an application that could eventually communicate with a real ESP32-based robotic prosthetic system.

Focus on functionality, clean architecture, thoughtful UI/UX, reliable state handling, and your approach to solving the problem.